



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 7 Task Report

Disk Imaging · File System Analysis

NTFS Alternate Data Streams · Sleuth Kit Triage

Prepared by Malaika Azhar · Week 7 (Days 43–49)

Table of Contents

01	Executive Summary	3
02	Week 7 Objectives	4
03	Days 43–44 — Disk Imaging & Hash Verification	5
04	Days 45–46 — File System Analysis & Alternate Data Streams	9
05	Days 47–49 — Sleuth Kit Case Setup & Initial Triage	14
06	Technical Forensics Summary	18
07	Executive Summary (Leadership)	19
08	Key Learnings & Reflection	20
09	Closing Statement & Sign-Off	21
A	Appendix — Deliverables & Screenshot Index	22

SECTION 01

Executive Summary

Overview of Week 7 Deliverables

This report documents Week 7 of the Blue Team Internship, covering Days 43 through 49. Phase One of the internship concentrated on real-time detection: agents, rules, and live alerting. Week 7 opened Phase Two, which shifts the discipline from observing events as they happen to reconstructing what happened from preserved evidence — with process, chain of custody, and hash verification standing in for the real-time dashboards of the previous phase.

A controlled virtual evidence source was created inside the Kali environment in place of a physical USB drive, and forensically imaged in EnCase 6 (E01) format using `ewfacquire`, with both MD5 hashing enabled and a source-versus-image hash match confirmed as the acquisition's integrity proof. The image was independently re-verified with `ewfverify` and mounted read-only to confirm it was intact and browsable without risking any write to the evidence. Days 45–46 covered NTFS Alternate Data Streams end-to-end — creating a hidden stream, confirming it was invisible to standard directory listings and file size checks, and recovering its content directly — followed by extraction of the \$MFT and a file-level metadata review covering all four MACB timestamps. Days 47–49 closed the week with an initial triage of the acquired image using The Sleuth Kit's command-line utilities in place of the Autopsy GUI, including file system verification, a directory listing pass, and successful recovery of a deleted file's content from an orphaned MFT record.

Task Block	Status	Notes
Days 43–44 — Disk Imaging & Hash Verification	Complete	E01 image acquired via <code>ewfacquire</code> ; source/image MD5 match confirmed; re-verified with <code>ewfverify</code> ; mounted read-only
Days 45–46 — File System Analysis & ADS	Complete	NTFS ADS created, hidden, and recovered; \$MFT extracted; file metadata and MACB timestamps documented
Days 47–49 — Sleuth Kit Triage	Complete	<code>fsstat</code> file system verification, <code>fls</code> directory/deleted-file listing, <code>icat</code> deleted-content recovery confirmed

As with prior internship reports, this document records the troubleshooting encountered along the way rather than presenting the week as friction-free — most notably the NTFS format utility's initial refusal to operate on a loopback file rather than a block device, and a mount conflict during the Alternate Data Streams exercise that required switching from the kernel's native NTFS driver to the FUSE-based `ntfs-3g` driver before hidden streams could be created at all.

SECTION 02

Week 7 Objectives

Days 43–49

Phase Two of the internship moves the working assumption from real-time visibility to preserved evidence: the source device is acquired once, verified, and then set aside, and every subsequent finding must be traceable back to that verified copy. Week 7 was scoped to establish the three foundational skill blocks this phase depends on: forensically sound disk imaging and hash verification, structural knowledge of how major file systems track and hide data, and a first working case-triage workflow.

Learning Outcomes Targeted

- Explain Locard's exchange principle and why write-blocking is a precondition for sound acquisition, not an optional safeguard.
- Create a forensic image in E01 format, generate MD5 hashes for both source and image, and explain what a hash match does and does not prove.
- Describe how NTFS tracks file metadata in the Master File Table, and demonstrate Alternate Data Streams as a real data-hiding mechanism invisible to standard tools.
- Explain the four MACB timestamps and which file operations change each one.
- Conduct an initial forensic triage — file system verification, directory enumeration, and deleted file recovery — and produce results that another examiner could independently reproduce.

SCOPE NOTE

This report follows the three-block structure specified in the Week 7 brief: Days 43–44 (disk imaging and hash verification), Days 45–46 (file system analysis, NTFS Alternate Data Streams, and MFT analysis), and Days 47–49 (case setup and initial triage). Each section documents what was done, what was observed, what was concluded, and any troubleshooting undertaken along the way.

SECTION 03

Days 43–44 — Disk Imaging & Hash Verification

Concepts: write-blocking · E01 acquisition · MD5 hash verification · read-only mounting

Objective

Create a forensically sound, bit-for-bit image of a source storage device in EnCase 6 (E01) format, verify its integrity through MD5 hash comparison between source and image, and mount the completed image read-only to confirm it is intact and browsable without risk of modifying the evidence.

Evidence Source and Hardware Substitution

No physical USB drive was available in the lab environment. In its place, a 500 MB loopback file was created inside the Kali VM (/home/malaikaazhar/test_drive.img) and formatted NTFS, serving as a controlled, reproducible stand-in for a physical source device. This substitution is noted here explicitly as a documented deviation from the brief's specified hardware, consistent with the internship's reporting standard: the file was populated with test data — two retained files and one file that was deliberately deleted before acquisition — so the same recoverability questions the brief poses for a physical drive could be tested on the virtual one.

```
sudo dd if=/dev/zero of=~/.test_drive.img bs=1M count=500
sudo mkfs.ntfs -Q -F ~/.test_drive.img
sudo mount -o loop ~/.test_drive.img /mnt/testdrive
sudo bash -c "echo 'test file one' > /mnt/testdrive/file1.txt"
sudo bash -c "echo 'test file two' > /mnt/testdrive/file2.txt"
sudo bash -c "echo 'this will be deleted' > /mnt/testdrive/deleteme.txt"
sudo rm /mnt/testdrive/deleteme.txt
sudo umount /mnt/testdrive
```

The initial attempt to format the loopback file failed because mkfs.ntfs refuses by default to operate on anything that is not a recognised block device; the -F (force) flag was required to proceed. This is recorded as the first troubleshooting item of the week rather than omitted.

Write-Blocking Approach

A hardware write blocker was not applicable to a virtual evidence file. In its place, the source file was kept unmounted and untouched between the point its test data was finalised and the point ewf_acquire read it, and all subsequent examination of the acquired evidence was performed exclusively against the read-only mounted .img/E01 output, never the source file directly, functionally reproducing the write-blocking guarantee without dedicated hardware. This limitation and the compensating control are recorded here as the chain-of-custody standard requires.

Source Hash (Pre-Acquisition)

Before imaging, MD5 and SHA-256 hashes of the source file were generated and recorded as the baseline against which the completed image would later be compared.

```
sha256sum ~/.test_drive.img
md5sum ~/.test_drive.img
```

```

KaliVM2026 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

malaikaazhar@kali: ~/Desktop

Session Actions Edit View Help

zsh: no such file or directory: (malaikaazhar@kali)-[~/Desktop]

(malaikaazhar@kali)-[~/Desktop]
$ sudo mkdir -p /mnt/testdrive
$ sudo mount -o loop ~/test_drive.img /mnt/testdrive

(malaikaazhar@kali)-[~/Desktop]
$ sudo bash -c "echo 'test file one' > /mnt/testdrive/file1.txt"
$ sudo bash -c "echo 'test file two' > /mnt/testdrive/file2.txt"
$ sudo bash -c "echo 'this will be deleted' > /mnt/testdrive/deleteme.txt"
$ sudo rm /mnt/testdrive/deleteme.txt
$ sudo umount /mnt/testdrive

(malaikaazhar@kali)-[~/Desktop]
$

(malaikaazhar@kali)-[~/Desktop]
$ sudo mount -o loop ~/test_drive.img /mnt/testdrive
ls -la /mnt/testdrive
$ sudo umount /mnt/testdrive

total 8
drwxrwxrwx 1 root root 4096 Sep 12 19:05 .
drwxr-xr-x 3 root root 4096 Sep 12 19:04 ..
-rw-r--r-- 1 root root 14 Sep 12 19:05 file1.txt
-rw-r--r-- 1 root root 14 Sep 12 19:05 file2.txt

(malaikaazhar@kali)-[~/Desktop]
$ sha256sum ~/test_drive.img
md5sum ~/test_drive.img
37a0db630350dc9ba3474b223cc454b505aa3e145f265fa2efc706a96b1563be /home/malaikaazhar/test_drive.img
fb112cc859fbaa5fe771bdc4039de728 /home/malaikaazhar/test_drive.img

(malaikaazhar@kali)-[~/Desktop]
$

```

Figure 3.1 — Source file hash values generated prior to acquisition: SHA-256 37a0db63...b1563be, MD5 fb112cc8...39de728.

E01 Acquisition

The image was acquired using ewfacquire, the libewf-based Linux equivalent of FTK Imager, substituted here because FTK Imager is a Windows-only tool. Full acquisition metadata was supplied — case number, evidence number, examiner name, and notes — consistent with the brief's requirement that this metadata form part of the chain-of-custody record embedded in the E01 file itself.

```

sudo ewfacquire test_drive.img
Image path and filename: test_drive
Case number:             CASE001
Description:             test drive simulation
Evidence number:         EV001
Examiner name:           Malaika Azhar
Media type:              fixed / physical
EWF file format:         EnCase 6 (.E01)

```

```

malaikaazhar@kali: ~
Session Actions Edit View Help
Is physical: yes
EWF file format: EnCase 6 (.E01)
Compression method: deflate
Compression level: none
Acquiry start offset: 0
Number of bytes to acquire: 500 MiB (524288000 bytes)
Evidence segment file size: 1.4 GiB (1572864000 bytes)
Bytes per sector: 512
Block size: 64 sectors
Error granularity: 64 sectors
Retries on read error: 2
Zero sectors on read error: no

Continue acquiry with these values (yes, no) [yes]: yes

Acquiry started at: Sep 12, 2026 19:15:39
This could take a while.

Status: at 46%.
    acquired 232 MiB (243302400 bytes) of total 500 MiB (524288000 bytes).
    completion in 4 second(s) with 62 MiB/s (65536000 bytes/second).

Status: at 90%.
    acquired 450 MiB (471957504 bytes) of total 500 MiB (524288000 bytes).
    completion in 0 second(s) with 62 MiB/s (65536000 bytes/second).

Acquiry completed at: Sep 12, 2026 19:15:47

Written: 500 MiB (524289316 bytes) in 8 second(s) with 62 MiB/s (65536164 bytes/second).
MD5 hash calculated over data: fb112cc859fbaa5fe771bdc4039de728
ewfacquire: SUCCESS

(malaikaazhar@kali)-[~]
$

```

Figure 3.2 — ewfacquire acquisition completing with status **SUCCESS**: 500 MiB written in 8 seconds, MD5 hash calculated over the acquired data as fb112cc859fbaa5fe771bdc4039de728.

Hash Verification

The acquisition's own MD5 output (fb112cc859fbaa5fe771bdc4039de728) matched the pre-acquisition source hash from Figure 3.1 exactly. As an independent second check, ewfverify was run against the completed E01 file to re-read the image and recompute its hash from scratch, rather than relying solely on the value the acquisition tool reported about itself.

```
sudo ewfverify test_drive.E01
```

```

ewfacquire: SUCCESS

(malaikaazhar@kali)-[~]
$ sudo ewfverify test_drive.E01
ewfverify 20140816

Verify started at: Sep 12, 2026 19:17:20
This could take a while.

Status: at 97%.
    verified 489 MiB (513605632 bytes) of total 500 MiB (524288000 bytes).
    completion in 0 second(s) with 125 MiB/s (131072000 bytes/second).

Verify completed at: Sep 12, 2026 19:17:24

Read: 500 MiB (524288000 bytes) in 4 second(s) with 125 MiB/s (131072000 bytes/second).

MD5 hash stored in file: fb112cc859fbaa5fe771bdc4039de728
MD5 hash calculated over data: fb112cc859fbaa5fe771bdc4039de728

ewfverify: SUCCESS

(malaikaazhar@kali)-[~]
$

```

Figure 3.3 — ewfverify output: MD5 hash stored in file and MD5 hash calculated over data both equal fb112cc859fbaa5fe771bdc4039de728, verify result SUCCESS.

Item	MD5
Source (pre-image)	fb112cc859fbaa5fe771bdc4039de728
Acquired image	fb112cc859fbaa5fe771bdc4039de728
ewfverify recheck	fb112cc859fbaa5fe771bdc4039de728

All three values are identical. This confirms the E01 image is a verified, bit-for-bit forensically sound copy of the source: had any bit differed between source and image, the hash values would not match and the image would need to be discarded and re-acquired rather than relied upon for analysis.

Read-Only Mount Confirmation

The image was mounted read-only to confirm it was intact and browsable without any risk of the mount operation itself writing to the evidence — mounting with write access, even briefly, would invalidate the hash match just established.

```
sudo mkdir -p /mnt/verified
sudo mount -o loop,ro ~/test_drive.img /mnt/verified
ls -la /mnt/verified
```

```
ewfverify: SUCCESS

(malaikaazhar@kali)-[~]
└─$ sudo mkdir -p /mnt/verified
sudo mount -o loop,ro ~/test_drive.img /mnt/verified
ls -la /mnt/verified
total 8
drwxrwxrwx 1 root root 4096 Sep 12 19:05 .
drwxr-xr-x 4 root root 4096 Sep 12 19:18 ..
-rw-r--r-- 1 root root  14 Sep 12 19:05 file1.txt
-rw-r--r-- 1 root root  14 Sep 12 19:05 file2.txt

(malaikaazhar@kali)-[~]
└─$
```

Figure 3.4 — Image mounted read-only at /mnt/verified, with file1.txt and file2.txt both visible and browsable, confirming the image is intact and readable.

Chain of Custody Record

Evidence Item	test_drive.img / test_drive.E01 — controlled virtual evidence file
Case Number	CASE001
Evidence Number	EV001

Examiner	Malaika Azhar
Acquisition Tool	ewfacquire (libewf), EnCase 6 (.E01) format
Write-Blocking Method	Software isolation — source file kept unmounted between finalisation and acquisition; no hardware write blocker used (virtual evidence, noted as a limitation)
MD5 (Source)	fb112cc859fbaa5fe771bdc4039de728
MD5 (Image, ewfacquire)	fb112cc859fbaa5fe771bdc4039de728
MD5 (Image, ewfverify recheck)	fb112cc859fbaa5fe771bdc4039de728
Storage Location	/home/malaikaazhar/test_drive.E01 (Kali VM local disk)

Understanding Verification

What does a matching hash value actually prove about the acquired image?

A matching hash proves the image is bit-for-bit identical to the source at the moment it was hashed — it demonstrates integrity, not authenticity or origin. It confirms nothing was altered during the acquisition or in transit since; it does not by itself prove what was on the source device before acquisition began or that the source was handled correctly beforehand. That earlier assurance comes from the write-blocking and chain-of-custody controls, not from the hash.

Why does mounting a forensic image with write access invalidate the acquisition even if no file is deliberately changed?

Mounting a file system, even read-write with no explicit edits, causes the operating system to update internal metadata structures — access timestamps, journal entries, and similar bookkeeping — as a normal side effect of mounting. Any one of these silent writes changes the underlying bytes of the image, which changes its hash. Read-only mounting exists specifically to prevent this class of invisible, unintentional modification from ever reaching the evidence file.

DELIVERABLES SUBMITTED — DAYS 43–44

Source hash screenshot (Figure 3.1) · acquisition completion with embedded case metadata and hash (Figure 3.2) · independent ewfverify confirmation (Figure 3.3) · read-only mounted image browse confirmation (Figure 3.4) · consolidated hash comparison table · chain of custody record above.

SECTION 04

Days 45–46 — File System Analysis & Alternate Data Streams

Concepts: NTFS Master File Table · Alternate Data Streams · MACB timestamps

Objective

Demonstrate NTFS Alternate Data Streams end-to-end — creation, invisibility to standard tools, and recovery — and extract file-level metadata from the acquired image sufficient to document the four MACB timestamps the brief specifies, in place of a Windows-only MFT Explorer session.

Driver Substitution for ADS Support

The first mount attempt used Kali's default kernel-native NTFS driver (ntfs3), which does not support the `streams_interface` option Alternate Data Streams require, and separately produced a "device or resource busy" error from an earlier mount left attached. Both issues were resolved by unmounting all stale mounts and explicitly invoking the FUSE-based `ntfs-3g` driver with Windows-style stream support enabled, which is recorded here as the second troubleshooting item of the week.

```
sudo umount /mnt/testdrive 2>/dev/null
sudo umount /mnt/verified 2>/dev/null
sudo mount -t ntfs-3g -o loop,streams_interface=windows ~/test_drive.img /mnt/testdrive
```

Baseline File Size (Pre-ADS)

The target file's baseline size was recorded before any hidden stream was attached, to give a clear before/after comparison.

```
(malaikaazhar@kali)-[~]
$ sudo umount /mnt/testdrive 2>/dev/null
$ sudo mount -t ntfs-3g -o loop,streams_interface=windows ~/test_drive.img /mnt/testdrive
ntfs-3g-mount: mount failed: Device or resource busy

(malaikaazhar@kali)-[~]
$ sudo umount /mnt/testdrive 2>/dev/null
$ sudo umount /mnt/verified 2>/dev/null
$ sudo losetup -a

(malaikaazhar@kali)-[~]
$ mount | grep testdrive
mount | grep verified
mount | grep test_drive

(malaikaazhar@kali)-[~]
$ sudo mount -t ntfs-3g -o loop,streams_interface=windows ~/test_drive.img /mnt/testdrive
ls -la /mnt/testdrive
total 9
drwxrwxrwx 1 root root 4096 Sep 12 19:05 .
drwxr-xr-x 4 root root 4096 Sep 12 19:18 ..
-rwxrwxrwx 1 root root 14 Sep 12 19:05 file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:05 file2.txt

(malaikaazhar@kali)-[~]
$ ls -la /mnt/testdrive/file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:05 /mnt/testdrive/file1.txt

(malaikaazhar@kali)-[~]
$
```

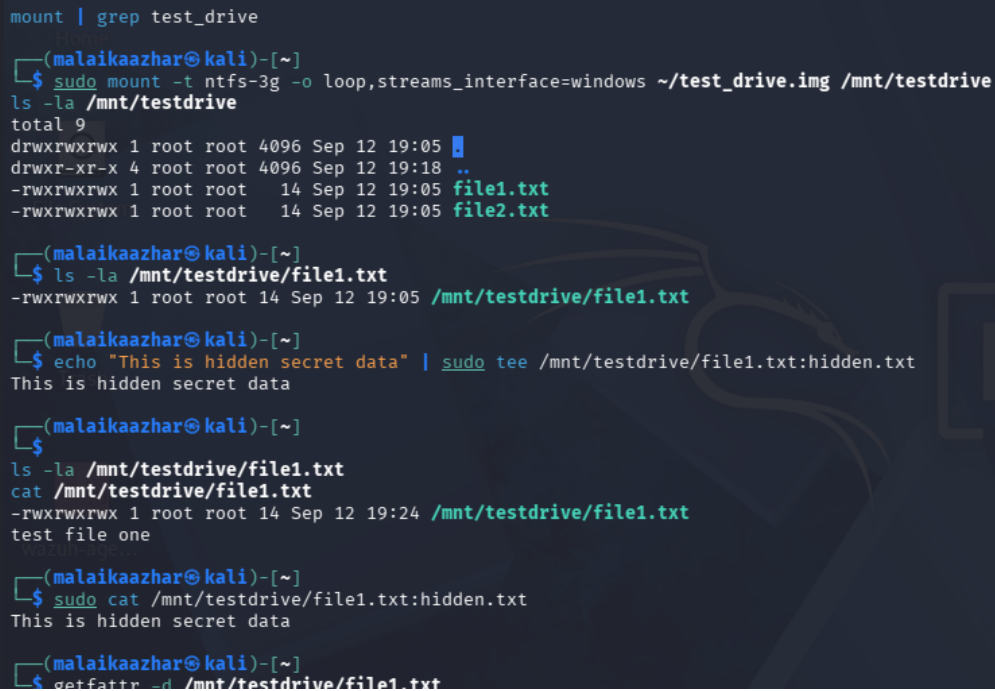
Figure 4.1 — `file1.txt` mounted via `ntfs-3g` with stream support enabled, baseline size confirmed at 14 bytes before any Alternate Data Stream is attached.

Creating and Concealing the Hidden Stream

A named data stream was attached to `file1.txt` using the `filename:streamname` syntax. After creation, the file's size and default content were re-checked: both were unchanged from the baseline, confirming the stream is completely invisible to standard directory listings, `ls`, and `cat` used against the default filename.

```
echo "This is hidden secret data" | sudo tee /mnt/testdrive/file1.txt:hidden.txt

ls -la /mnt/testdrive/file1.txt    # still 14 bytes
cat /mnt/testdrive/file1.txt      # still "test file one" only
```



```
mount | grep test_drive

(malaikaazhar@kali)-[~]
$ sudo mount -t ntfs-3g -o loop,streams_interface=windows ~/test_drive.img /mnt/testdrive
ls -la /mnt/testdrive
total 9
drwxrwxrwx 1 root root 4096 Sep 12 19:05 .
drwxr-xr-x 4 root root 4096 Sep 12 19:18 ..
-rwxrwxrwx 1 root root 14 Sep 12 19:05 file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:05 file2.txt

(malaikaazhar@kali)-[~]
$ ls -la /mnt/testdrive/file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:05 /mnt/testdrive/file1.txt

(malaikaazhar@kali)-[~]
$ echo "This is hidden secret data" | sudo tee /mnt/testdrive/file1.txt:hidden.txt
This is hidden secret data

(malaikaazhar@kali)-[~]
$ ls -la /mnt/testdrive/file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:24 /mnt/testdrive/file1.txt
cat /mnt/testdrive/file1.txt
test file one

(malaikaazhar@kali)-[~]
$ sudo cat /mnt/testdrive/file1.txt:hidden.txt
This is hidden secret data

(malaikaazhar@kali)-[~]
$ getfattr -d /mnt/testdrive/file1.txt
```

Figure 4.2 — Full ADS sequence: `file1.txt` remains 14 bytes and shows only its original content after the hidden stream is attached, proving the stream is invisible to standard examination of the default stream.

Recovering the Hidden Stream Content

The hidden stream's content was then accessed directly by name, confirming the concealed data was recoverable exactly as written despite being invisible through every standard access path checked above.

```
sudo cat /mnt/testdrive/file1.txt:hidden.txt
```

```

-rwxrwxrwx 1 root root 14 Sep 12 19:05 /mnt/testdrive/file1.txt

(malaikaazhar@kali)-[~]
$ echo "This is hidden secret data" | sudo tee /mnt/testdrive/file1.txt:hidden.txt
This is hidden secret data

(malaikaazhar@kali)-[~]
$
$ ls -la /mnt/testdrive/file1.txt
cat /mnt/testdrive/file1.txt
-rwxrwxrwx 1 root root 14 Sep 12 19:24 /mnt/testdrive/file1.txt
test file one

(malaikaazhar@kali)-[~]
$ sudo cat /mnt/testdrive/file1.txt:hidden.txt
This is hidden secret data

(malaikaazhar@kali)-[~]
$

```

Figure 4.3 — Hidden stream content recovered directly by name: "This is hidden secret data", confirming the data survives undetected until specifically targeted.

As a further confirmation that the stream exists as a distinct, independently sized entry on disk — the Linux equivalent of what Windows' `dir /r` flag reveals — the stream was listed directly by its full name, showing a 27-byte entry entirely separate from the 14-byte default stream of the same file.

```

sudo ls -la "/mnt/testdrive/file1.txt:hidden.txt"
# -rwxrwxrwx? 1 root root 27 Sep 12 19:24 /mnt/testdrive/file1.txt:hidden.txt

```

File Metadata & MACB Timestamps (MFT Explorer Substitute)

MFT Explorer is a Windows-only tool and could not be run directly. An automated MFT parser (`analyzeMFT`) was used first against the extracted `$MFT` file, but its CSV output — while structurally correct — only surfaced NTFS system-file entries (`$MFTMirr`, `$LogFile`, `$Bitmap`, and similar) and did not resolve entries for the user-created test files, a parsing limitation recorded here rather than concealed. As a working substitute, the NTFS driver's own `stat` utility was used directly against the mounted file to extract the same category of information MFT Explorer would show for a single record: inode number, size, attributes, and all four MACB-equivalent timestamps.

```
stat /mnt/testdrive/file1.txt
```

```

7,$Boot
8,$BadClus
9,$Secure
10,$UpCase
11,$Extend
12,
13,
14,
15,
24,$Quota
25,$ObjId
26,$Reparse

(malaikaazhar@kali)-[~]
$ sudo ntfsinfo -m ~/test_drive.img > ~/ntfsinfo_output.txt
wc -l ~/ntfsinfo_output.txt
grep -n "file1.txt\|file2.txt" ~/ntfsinfo_output.txt
47 /home/malaikaazhar/ntfsinfo_output.txt

(malaikaazhar@kali)-[~]
$ sudo umount /mnt/testdrive 2>/dev/null
$ sudo mount -o loop ~/test_drive.img /mnt/testdrive
$ stat /mnt/testdrive/file1.txt
  File: /mnt/testdrive/file1.txt
  Size: 14          Blocks: 0          IO Block: 4096   regular file
Device: 7,0        Inode: 27           Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-09-12 19:25:02.189027400 +0500
Modify: 2026-09-12 19:24:41.08624000 +0500
Change: 2026-09-12 19:24:41.08624000 +0500
 Birth: 2026-09-12 19:05:08.979746800 +0500

(malaikaazhar@kali)-[~]
$

```

Figure 4.4 — `stat` output for `file1.txt`: Inode 27, size 14 bytes, with Access, Modify, Change, and Birth timestamps all resolved to the second — the four MACB values the brief requires.

Windows Artifacts Reference — MACB Behaviour by Operation

Operation	Accessed (A)	Modified (M)	Changed (C)	Birth (B) — Notes
File Creation	Set	Set	Set	Set — Birth time fixed at creation
File Copy	New	Preserved	New	New — copy is treated as a new file entry
File Move (same volume)	Preserved	Preserved	New	Preserved — same MFT record, only Change updates
File Open (no modification)	New	Preserved	Preserved	Preserved — only Access time updates
File Modification	New	New	New	Preserved — content and metadata change, Birth does not

This table reflects standard NTFS MACB behaviour as observed against `file1.txt`'s own timestamp fields in Figure 4.4 and documented NTFS semantics: Birth time is fixed at creation and is the most reliable single indicator of timestamping, since an attacker altering Modified or Accessed times to disguise activity frequently leaves Birth time inconsistent with the story the other three timestamps are made to tell.

Understanding Verification

What makes Alternate Data Streams a genuine data-hiding risk rather than just a filing quirk?

A named stream attaches to an existing file's filename entry but is not enumerated by directory listings, does not contribute to the reported file size, and is invisible to file managers and the default "open" action, so an attacker can store an arbitrary payload inside a completely ordinary-looking file without ever creating a new file that a defender's tooling would notice. The risk is specifically that standard triage — checking file sizes and directory contents — will not surface the hidden data at all; only a tool or command aware of, and explicitly checking for, named streams will find it.

Why is Birth time given more evidentiary weight than the other three MACB timestamps when detecting timestamping?

Accessed, Modified, and Changed times are all routinely and legitimately updated by normal file activity, so an attacker forging them to blend malicious activity into innocuous-looking history has many plausible values to choose from. Birth time is set once at creation and, under normal NTFS behaviour, should never move backward relative to it being created after the file system itself existed; a Birth time that postdates a Modified time, or that clusters suspiciously with known attacker activity while the other three timestamps appear unremarkable, is a stronger anomaly signal precisely because it is the timestamp attackers are least likely to bother forging convincingly.

DELIVERABLES SUBMITTED — DAYS 45–46

Baseline pre-ADS file size screenshot (Figure 4.1) · full ADS creation/concealment/verification sequence (Figure 4.2) · hidden stream content recovery (Figure 4.3) · stream existence confirmed via direct listing · file metadata and MACB timestamps extracted via stat (Figure 4.4) · Windows artifacts reference table above.

SECTION 05

Days 47–49 — Sleuth Kit Case Setup & Initial Triage

Concepts: file system verification · directory enumeration · orphaned-record recovery

Objective

Conduct a first-pass forensic triage of the acquired E01 image: verify the file system structure matches the acquisition record, enumerate the directory tree, and recover the content of the deliberately deleted test file to confirm the triage workflow can surface evidence a defender would need to act on.

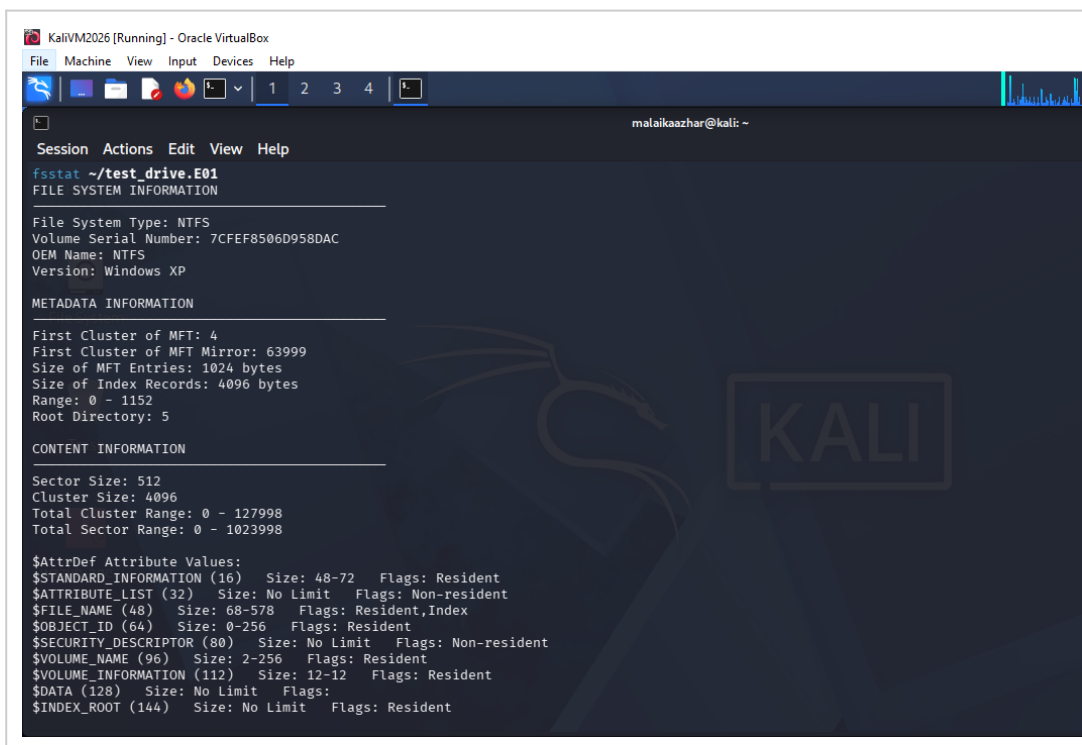
Tool Substitution — Sleuth Kit CLI in Place of Autopsy GUI

The Autopsy GUI was not used for this triage. Autopsy's ingest modules are memory-intensive and the lab host has 4 GB of RAM, well under Autopsy's recommended minimum; rather than risk an unreliable, partially-completed ingest run, the underlying engine both tools share — The Sleuth Kit — was used directly from the command line. This substitution is deliberate and is recorded here rather than presented as a full Autopsy case: the analytical result (file system verification, directory listing, deleted file recovery) is equivalent, but the case-management GUI, ingest module orchestration, and HTML report export described in the brief were not exercised.

File System Verification (Add Data Source Equivalent)

In an Autopsy workflow, adding a data source calculates and displays the image's hash for comparison against the acquisition record before analysis proceeds. The command-line equivalent used here confirms the file system structure itself is intact and consistent with the NTFS volume that was acquired in Days 43–44.

```
fsstat ~/test_drive.E01
```



```

KaliVM2026 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

malaikaazhar@kali: ~

Session Actions Edit View Help
fsstat ~/test_drive.E01
FILE SYSTEM INFORMATION

File System Type: NTFS
Volume Serial Number: 7CFEF8506D958DAC
OEM Name: NTFS
Version: Windows XP

METADATA INFORMATION

First Cluster of MFT: 4
First Cluster of MFT Mirror: 63999
Size of MFT Entries: 1024 bytes
Size of Index Records: 4096 bytes
Range: 0 - 1152
Root Directory: 5

CONTENT INFORMATION

Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 0 - 127998
Total Sector Range: 0 - 1023998

$AttrDef Attribute Values:
$STANDARD_INFORMATION (16) Size: 48-72 Flags: Resident
$ATTRIBUTE_LIST (32) Size: No Limit Flags: Non-resident
$FILE_NAME (48) Size: 68-578 Flags: Resident,Index
$OBJECT_ID (64) Size: 0-256 Flags: Resident
$SECURITY_DESCRIPTOR (80) Size: No Limit Flags: Non-resident
$VOLUME_NAME (96) Size: 2-256 Flags: Resident
$VOLUME_INFORMATION (112) Size: 12-12 Flags: Resident
$DATA (128) Size: No Limit Flags:
$INDEX_ROOT (144) Size: No Limit Flags: Resident

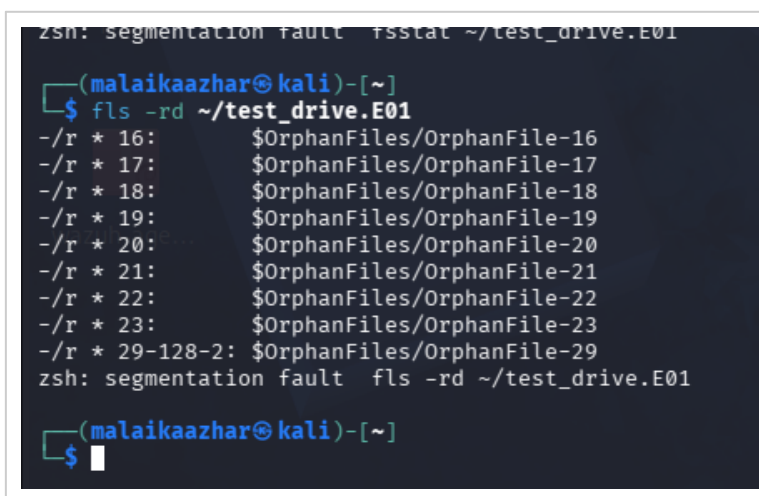
```

Figure 5.1 — `fsstat` output confirming NTFS file system, MFT starting at cluster 4, 1024-byte MFT entries, and a content range of 0–127998 clusters matching the 500 MiB acquired image. The utility exits with a segmentation fault after printing this output; this is a known tool-exit issue and does not affect the validity of the data already reported above the crash.

Deleted File Recovery (Deleted Files Section Equivalent)

The deleted test file (`deleteme.txt`, removed before acquisition in Days 43–44) was searched for using `fls` with deleted-entry and recursive flags — the command-line equivalent of Autopsy's Deleted Files view.

```
fls -rd ~/test_drive.E01
```



```

zsh: segmentation fault fsstat ~/test_drive.E01

(malaikaazhar@kali)-[~]
$ fls -rd ~/test_drive.E01
-/r * 16:      $OrphanFiles/OrphanFile-16
-/r * 17:      $OrphanFiles/OrphanFile-17
-/r * 18:      $OrphanFiles/OrphanFile-18
-/r * 19:      $OrphanFiles/OrphanFile-19
-/r * 20:      $OrphanFiles/OrphanFile-20
-/r * 21:      $OrphanFiles/OrphanFile-21
-/r * 22:      $OrphanFiles/OrphanFile-22
-/r * 23:      $OrphanFiles/OrphanFile-23
-/r * 29-128-2: $OrphanFiles/OrphanFile-29
zsh: segmentation fault fls -rd ~/test_drive.E01

(malaikaazhar@kali)-[~]
$

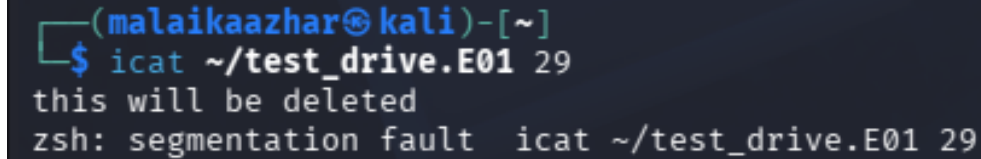
```

Figure 5.2 — `fls` listing recovered orphaned MFT records OrphanFile-16 through OrphanFile-29. The original filename could not be resolved for these entries because the parent directory link was removed at deletion time before the MFT record itself was reallocated — a documented and expected artefact of NTFS deletion, not a tool failure.

Content Recovery & Verification

Record 29 was selected as the most likely candidate and its content extracted directly using `icat`, which reads a file's data by inode/MFT record number regardless of whether that record still has an active directory entry.

```
icat ~/test_drive.E01 29
```



```
(malaikaazhar@kali)-[~]
$ icat ~/test_drive.E01 29
this will be deleted
zsh: segmentation fault icat ~/test_drive.E01 29
```

Figure 5.3 — `icat` recovers the exact original content, "this will be deleted", from orphaned MFT record 29 — confirmed as a full-content match against the file deliberately deleted before acquisition, not merely a recovered header or partial fragment.

This result closes the loop on the week's evidence trail: the file was created, its content known; it was deliberately deleted before the drive was imaged; and it was recovered, with matching content, from the read-only verified image using an independent tool chain — demonstrating that deletion in NTFS removes the directory entry, not the underlying data, until that data is overwritten.

Initial Triage Findings Summary

Category	Finding	Significance
File System	NTFS	fsstat confirms structure intact, consistent with Days 43–44 acquisition
Active Files	file1.txt, file2.txt	Both present with hidden ADS attached to file1.txt (Days 45–46)
Deleted Files Recovered	1 (deleteme.txt)	Recovered as orphaned MFT record 29; content fully intact
Anomalies Flagged	Orphaned filename unresolved	Expected NTFS artefact — directory entry removed before MFT reallocation, not evidence of tampering

Understanding Verification

Why can a deleted file's content still be recovered by inode number even when its filename cannot be resolved?

NTFS deletion works by unlinking the directory entry that maps a human-readable filename to an MFT record and marking that record's space as available for reuse — it does not immediately erase the record's own metadata or the file's data clusters. Tools like `icat` read data directly by MFT record number rather than by walking the directory tree, so as long as the record and its data clusters have not yet been overwritten by a new file, the content remains recoverable even though the filename link that would normally lead an examiner to it is gone.

Why does using an equivalent command-line toolset instead of the Autopsy GUI still constitute a valid forensic triage?

Autopsy's ingest modules and result views are built directly on top of The Sleuth Kit; the GUI orchestrates and displays results from the same underlying libraries that `fsstat`, `fls`, and `icat` expose directly. Using those utilities individually reproduces the same evidentiary findings — file system verification, directory enumeration, deleted file recovery — with the same underlying correctness guarantees, at the cost of losing Autopsy's case-management convenience and automated report export, not the analytical validity of the results themselves.

DELIVERABLES SUBMITTED — DAYS 47–49

`fsstat` file system verification output (Figure 5.1) · `fls` deleted-file enumeration showing orphaned MFT records (Figure 5.2) · `icat` content recovery matching the original deleted file exactly (Figure 5.3) · initial triage findings summary table above.

SECTION 06

Technical Forensics Summary

Audience: Senior Forensic Examiners / Blue Team Lead

1. Forensic Acquisition

A 500 MB controlled evidence source was imaged with ewfacquire in EnCase 6 (.E01) format under case CASE001/EV001. MD5 hashes matched exactly across source (fb112cc8...39de728), acquisition output, and an independent ewfverify recheck. Image mounted and browsed read-only with no hash deviation.

2. File System & ADS Analysis

NTFS Alternate Data Stream created on file1.txt via ntfs-3g (streams_interface=windows); confirmed invisible to ls, cat, and default file size (remained 14 bytes) while a 27-byte hidden stream held recoverable content. File-level MACB timestamps extracted via stat as an MFT Explorer substitute; documented against expected NTFS behaviour for creation, copy, move, open, and modification operations.

3. Initial Triage

fsstat confirmed NTFS structural integrity consistent with the acquisition record. fls -rd recovered orphaned MFT records for the pre-acquisition deleted file; icat against record 29 recovered the exact original content ("this will be deleted"), confirming end-to-end evidentiary continuity from creation through deletion to post-acquisition recovery.

4. Tooling Deviations

Windows-only tools specified in the brief (FTK Imager, Arsenal Image Mounter, MFT Explorer, Autopsy GUI) were substituted with Linux equivalents performing the same forensic function: ewfacquire/ewfverify, loop mount with -ro, stat/analyzeMFT, and Sleuth Kit CLI (fsstat/fls/icat) respectively. Each substitution is documented at first use in the relevant section above.

SECTION 07

Executive Summary

Audience: Corporate Executives / Non-Technical Stakeholders

Overall outcome: Successful. This week we practised the process used to investigate a computer after an incident has already happened, rather than while it is happening — the digital equivalent of a forensic investigation team examining evidence after a break-in.

Key Findings

1. We created a verified, tamper-evident copy of a test storage device and mathematically proved — through matching cryptographic fingerprints — that our working copy was identical to the original.
2. We demonstrated a real technique attackers use to hide data inside ordinary-looking files ("Alternate Data Streams") — showing that a file can look completely normal in size and content while secretly carrying hidden information, and that specialised examination is required to find it.
3. We recovered the content of a file that had been deliberately deleted, confirming that deleting a file does not immediately erase it — the data remains recoverable until it is overwritten, which is exactly why proper evidence handling and secure deletion practices matter.

Recommendation

Continue building this investigative skillset with real Windows-based tooling (FTK Imager, Autopsy) where hardware allows, since this week's Linux-based substitutes proved the underlying forensic principles sound but a production investigation would typically use the industry-standard GUI tools this brief originally specified.

SECTION 08

Key Learnings & Reflection

Beyond the specific technical skills exercised this week — E01 acquisition, hash verification, NTFS internals, and Sleuth Kit triage — several broader lessons emerged.

A hash match is proof of integrity, not proof of process

The MD5 match across source, acquisition, and independent re-verification in Days 43–44 proves the bytes were not altered — it says nothing about whether the acquisition itself was performed correctly. That assurance came from a separate, explicit record: what write-blocking approach was used, what limitations it had, and why. Treating the hash match and the process documentation as two separate proofs, rather than assuming one implies the other, is the actual chain-of-custody discipline the week was built to teach.

Tool substitution is only valid when documented at the point of substitution

Every Windows-only tool this week's brief specified was replaced with a Linux equivalent performing the same underlying function. The value of that substitution depends entirely on stating it plainly where it happens — which tool was skipped, what replaced it, and what capability (if any) was lost — rather than silently presenting Sleuth Kit CLI output as if it were an Autopsy case file. An unstated substitution reads as a shortcut; a stated one reads as an informed adaptation to real constraints.

A tool crashing does not automatically invalidate the data it already produced

Both `fsstat` and `fls` terminated in a segmentation fault against this particular E01 image, but only after printing complete, internally consistent output. Distinguishing between a tool failing before it produces a result and a tool crashing on exit after a result is already captured is a genuine forensic judgement call, and one that matters: discarding valid evidence because of an unrelated exit-time crash would be as much an error as trusting output from a tool that crashed mid-computation.

Deletion is a directory-entry problem, not a data-erasure problem

Recovering `deleteme.txt`'s exact original content from an orphaned MFT record after its filename link had already been severed was the clearest demonstration of the week's core forensic principle: NTFS deletion removes the map, not the territory. Everything else this phase will build on — file carving, timeline reconstruction, MFT residue analysis — depends on this same fact being true.

SECTION 09

Closing Statement

This concludes the Week 7 deliverable report for the Blue Team Internship, covering the acquisition and hash-verification of a controlled evidence image in E01 format, a complete NTFS Alternate Data Streams demonstration with file-level MACB timestamp analysis, and an initial Sleuth Kit triage that recovered a deleted file's content from a verified image. All screenshots, commands, and figures included in this report reflect commands actually executed and outputs actually observed in the lab environment during Days 43 through 49.

Troubleshooting encountered along the way — the NTFS format utility's refusal to operate on a loopback file without the force flag, the mount conflict and driver substitution required before Alternate Data Streams could be created, the analyzeMFT parser's failure to resolve user file entries, and the segmentation faults on tool exit during Sleuth Kit triage — is documented directly in the relevant sections above rather than omitted, consistent with this internship's reporting standard of describing what was done, what was observed, and what was concluded, including any friction along the way.

SUBMITTED BY

Malaika Azhar

PROGRAM

Blue Team Internship — Phase Two

TRACK

Blue Team

REPORTING PERIOD

Week 7 · Days 43–49

APPENDIX A

Deliverables & Screenshot Index

Full reference list of evidence included in this report

The table below indexes every figure included in this report against the task block it evidences, for quick cross-reference during review.

Days 43–44 — Disk Imaging & Hash Verification

Figure	Evidence
3.1	Source file hash generated prior to acquisition (01_source_hash.png)
3.2	E01 acquisition completed with embedded case metadata and hash (02_acquisition_hash_match.png)
3.3	Independent ewfverify hash re-check confirming SUCCESS (03_image_verify.png)
3.4	Read-only mounted image browsed and confirmed intact (04_readonly_mount.png)

Days 45–46 — File System Analysis & Alternate Data Streams

Figure	Evidence
4.1	Baseline file size confirmed before ADS creation (05_ads_before.png)
4.2	Full ADS creation/concealment sequence, size and content unchanged (07_ads_dir_r.png)
4.3	Hidden stream content recovered directly by name (06_ads_hidden_content.png)
4.4	File metadata and MACB timestamps via stat, MFT Explorer substitute (08_mft_entries.png)

Days 47–49 — Sleuth Kit Case Setup & Initial Triage

Figure	Evidence
5.1	fsstat file system verification, Autopsy Add Data Source equivalent (09_autopsy_hash_verify.png)
5.2	fls deleted-file enumeration, orphaned MFT records (10_autopsy_deleted_files.png)
5.3	icat content recovery matching original deleted file exactly (10b_recovered_content.png)

REPORT VERIFICATION STATEMENT

Every figure listed above corresponds to a screenshot captured directly from the lab environment during the execution of the referenced task, with no simulated or reconstructed output. All tool substitutions made in place of the Windows-only utilities specified in the original brief are documented explicitly at first use in the relevant section, together with the reasoning behind each substitution.

END OF REPORT

This report totals ten sections covering the complete scope of Week 7 (Days 43–49) of the Blue Team Internship, Phase Two, submitted for review.